

Analisi e Progettazione del Software

Dai requisiti alla progettazione

Oliviero Riganelli

Fare la cosa giusta, e fare la cosa bene, iterativamente

- L'**analisi** dei requisiti e l'analisi a oggetti sono interessate ad imparare a **fare la cosa giusta**
- Il lavoro successivo di **progettazione** porrà l'accento sul **fare la cosa bene**
- Nello sviluppo iterativo
 - in ciascuna iterazione vengono svolte attività di analisi dei requisiti, analisi e progettazione orientata agli oggetti, implementazione, test, ...
 - *enfasi via via maggiore su progettazione e implementazione*

Provocare il cambiamento all'inizio

E naturale e salutare scoprire e modificare alcuni requisiti durante il lavoro di progettazione e di implementazione, soprattutto nelle fasi iniziali

- I metodi iterativi ed evolutivi abbracciano il cambiamento
- Si cerca però di provocare il cambiamento nelle iterazioni iniziali in modo da avere degli obiettivi più stabili per le iterazioni successive

Il processo di analisi e modellazione non richiede settimane di lavoro?

- Il tempo richiesto per svolgere l'analisi vera e propria esaminata finora è realisticamente di alcune **ore** o **giorni**
- Dall'inizio del progetto potrebbero però essere trascorse alcune settimane di preparazione per svolgere attività quali la ricerca di risorse, la pianificazione, l'allestimento dell'ambiente ...

Analisi e Progettazione del Software

Architettura Logica

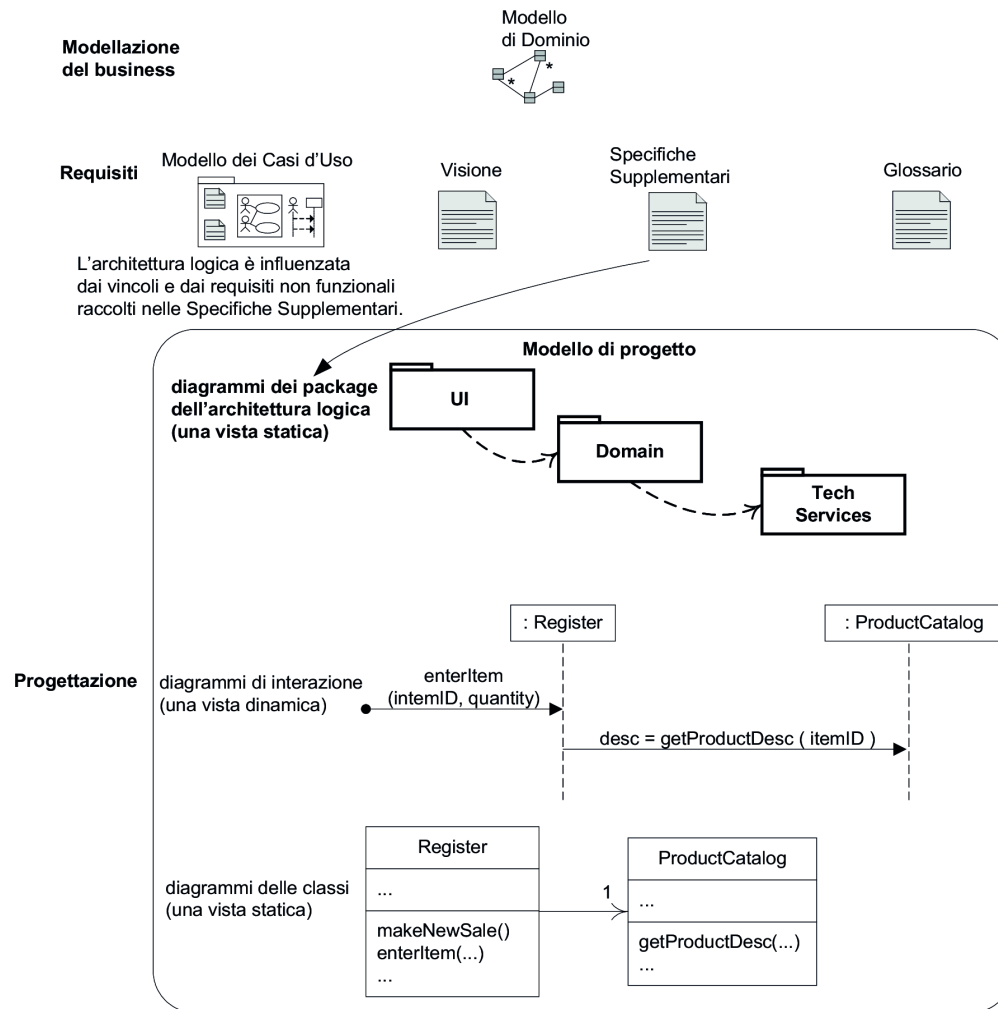
Oliviero Riganelli

Introduzione

- Passando da un lavoro di analisi alla progettazione del software si inizia a pensare su larga scala
- La progettazione di un tipico sistema orientato agli oggetti è basata su diversi strati architetturali
 - Come uno strato l'interfaccia utente, uno strato della logica applicativa e così via

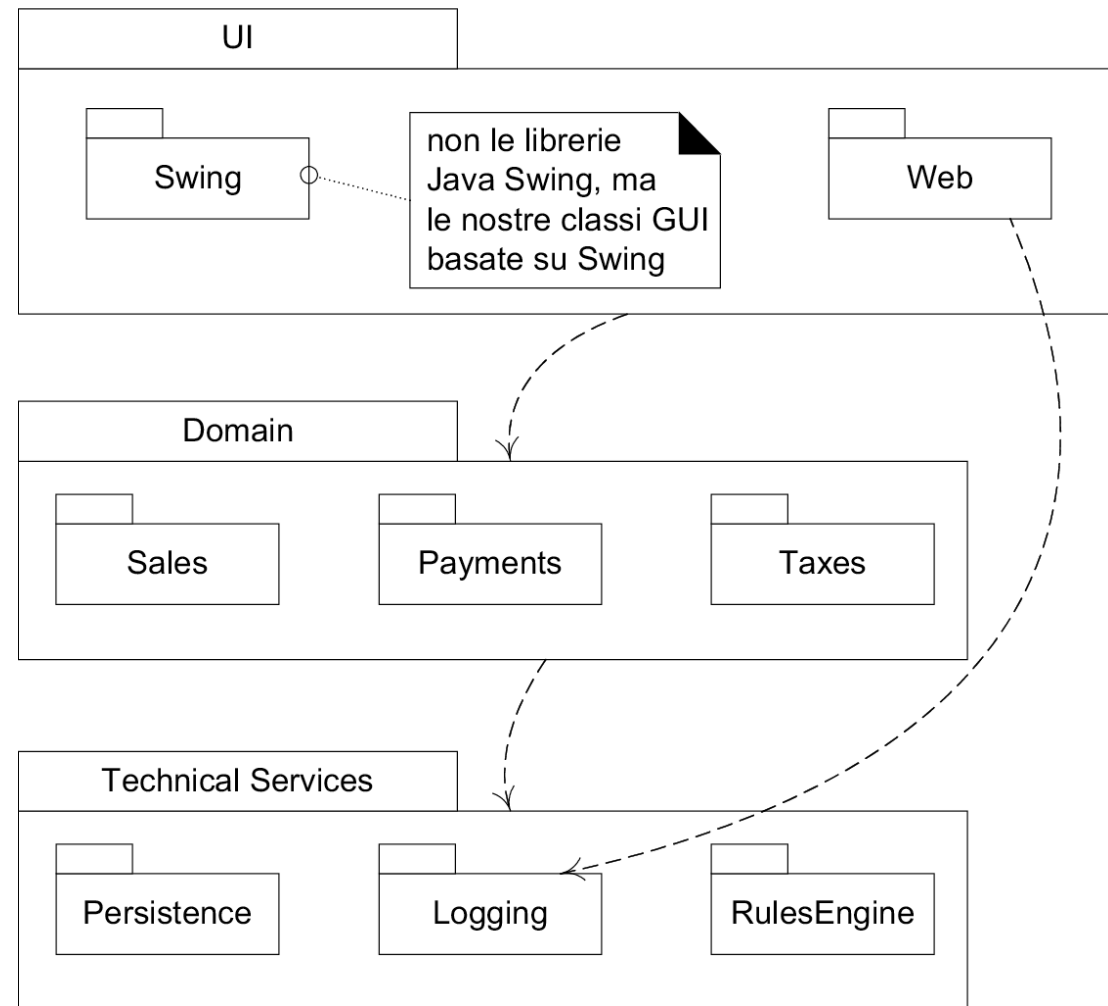
Influenza tra alcuni elaborati di UP

Alcune relazioni tra gli elaborati di UP



Che cosa è l'architettura logica

- L'**architettura logica** di un sistema software è l'organizzazione su larga scala delle classi software in package (o namespace), sottoinsiemi e strati
- Uno stile comune per l'architettura logica è l'**architettura a strati**



Architettura a strati

- Uno **strato** è un gruppo a grana molto grossa di classi, package o sottosistemi, che ha delle responsabilità coese rispetto a un aspetto importante il sistema
 - *Gli strati più alti ricorrono ai servizi degli strati più bassi (normalmente non avviene il contrario)*
- Gli strati di un'applicazione software comprendono normalmente:
 - *presentazione – interfaccia utente*
 - *logica applicativa o strato del dominio – elementi che rappresentano concetti del dominio*
 - *servizi tecnici*

Architettura a strati

- In un'**architettura strati stretta**, uno strato può solo richiamare i servizi dello strato immediatamente sottostante
- In un'**architettura a strati rilassata** uno strato più alto può richiamare i servizi di strati più bassi di diversi livelli

Gli strati di interesse per gli studi di caso

- Il corso si concentra sullo ***strato della logica applicativa*** (o ***strato del dominio***) con alcune discussioni secondari sugli altri strati
- Le lezioni di progettazione orientata agli oggetti apprese nel contesto dello strato della logica applicativa sono applicabili a tutti gli altri strati o componenti

Che cosa è un'architettura software

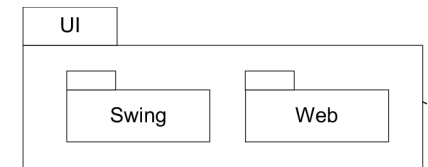
Una architettura software è

- l'insieme delle decisioni significative sull'organizzazione di un sistema software,
- la scelta degli elementi strutturali da cui è composto il sistema e delle relative interfacce,
- insieme al loro comportamento specificato dalla collaborazione tra questi elementi,
- la composizione di questi elementi strutturali e comportamentali in sottosistemi via via più ampi,
- e lo stile architetturale che guida questa organizzazione

L'architettura software ha a che fare con la larga scala

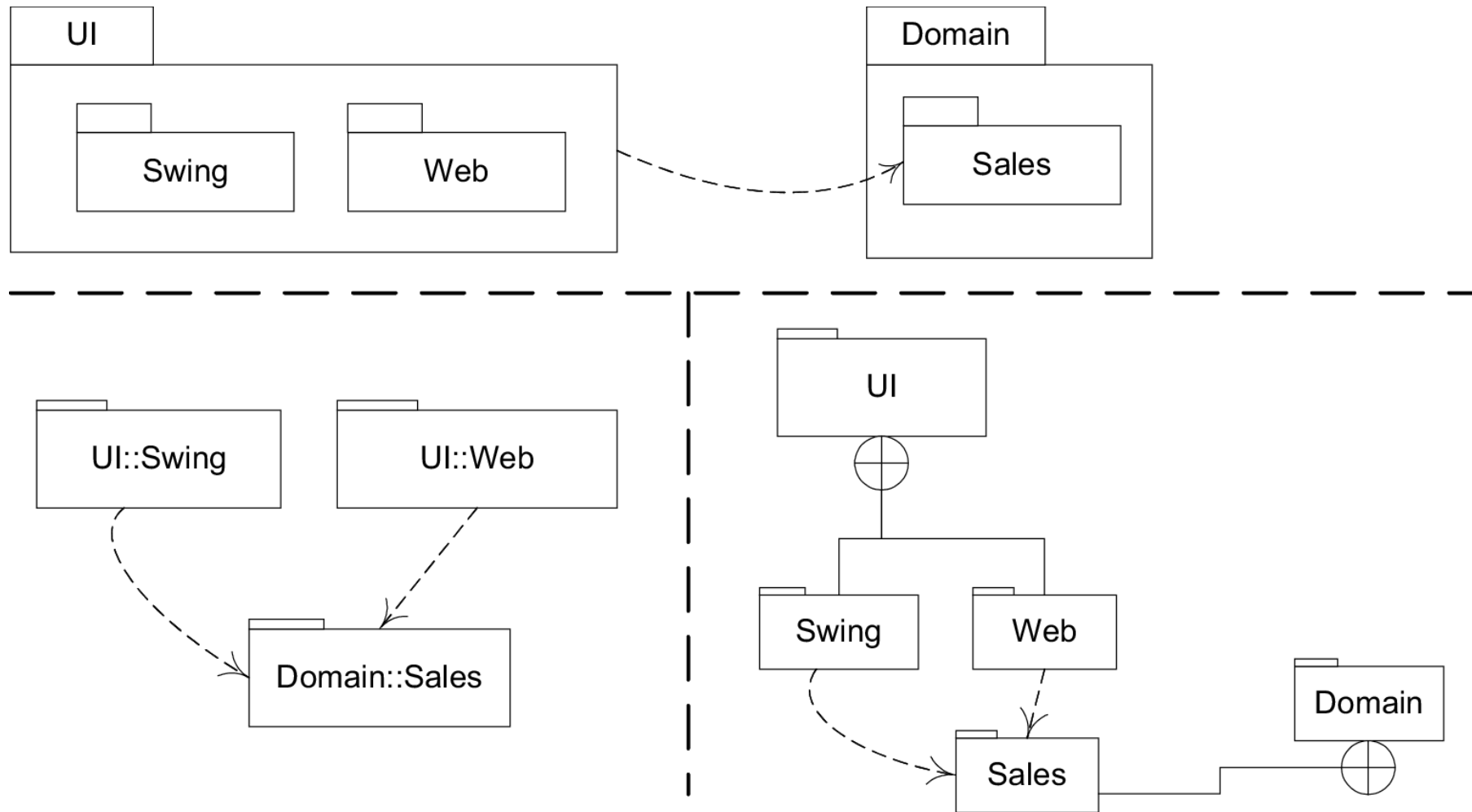
Diagrammi dei Package

- L'architettura logica può essere illustrata mediante un diagramma dei package di UML
 - *Uno strato può essere modellato come un package UML*
- Un package UML può raggruppare qualunque cosa (classi, altri package, casi d'uso ...)
 - *È molto comune l'**annidamento** di package*
 - *Per mostrare la dipendenza tra i package viene utilizzata una **dipendenza UML***
 - *Un package UML rappresenta un **namespace** cosicché è possibile definire due classi con lo stesso nome su package diversi*



`java::util::Date`

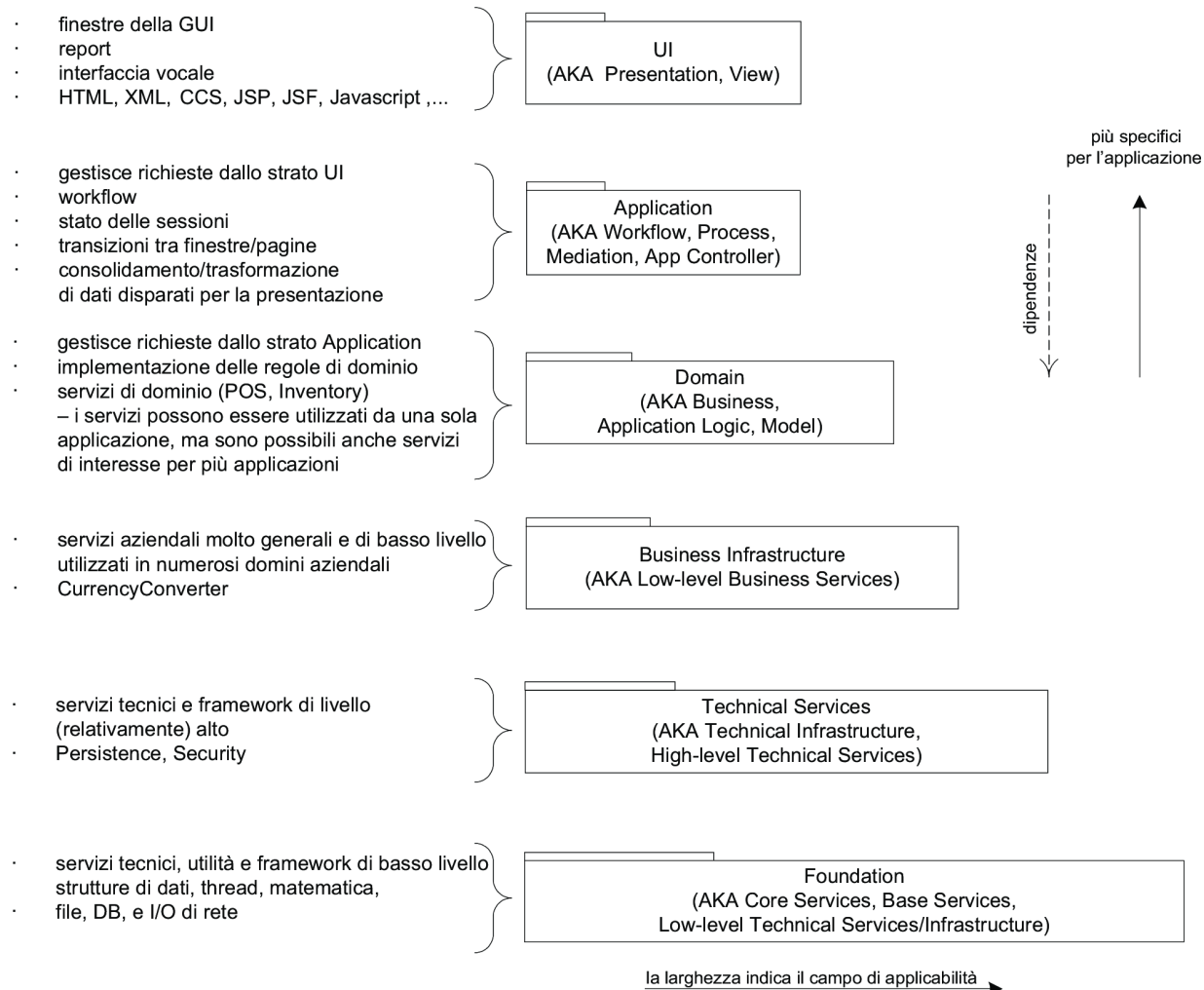
Come mostrare l'annidamento dei package



Progettazione degli strati

- Organizzare la struttura logica di un sistema in strati separati con responsabilità distinte e correlate, con una separazione netta e coesa degli interessi
 - *gli strati inferiori sono servizi generali e di basso livello,*
 - *gli strati superiori sono più specifici per l'applicazione*
- Collaborazioni e accoppiamenti vanno dagli strati più alti a quelli più bassi
- L'obiettivo è la suddivisione di un sistema in un insieme di elementi software che, per quanto possibile, possano essere sviluppati e modificati ciascuno indipendentemente dagli altri

Scelta comune per gli strati



Vantaggi dell'uso a strati

- C'è una separazione degli interessi, la separazione tra servizi di alto e di basso livello e tra servizi generali e quelli più specifici per l'applicazione
 - *Questo riduce l'accoppiamento e le dipendenze, migliora la coesione, aumenta la possibilità di riuso e aumenta la chiarezza*
- La complessità relativa a questi aspetti è incapsulata e può essere decomposta
- Alcuni strati possono essere sostituiti da nuove implementazioni
- Gli strati più bassi contengono funzioni riusabili
- Alcuni strati possono essere distribuiti
- Lo sviluppo da parte dei team è favorito dalla segmentazione logica

Responsabilità coese e separazione degli interessi

In uno strato, le responsabilità degli oggetti devono essere fortemente coese l'uno all'altro, e non devono essere mescolate con le responsabilità degli altri strati, esempio:

- Gli oggetti della UI devono essere incentrati sulle attività della UI (creare finestre e widget, catturare eventi UI ...)
- Gli oggetti dello strato della logica applicativa (o del dominio) devono essere incentrati sulla logica applicativa, come calcolare il totale o le imposte per una vendita
- Gli oggetti della UI non devono implementare logica applicativa, come calcolare le imposte
- Le classi della logica applicativa non devono catturare gli eventi UI del mouse o della tastiera

Organizzazione del codice

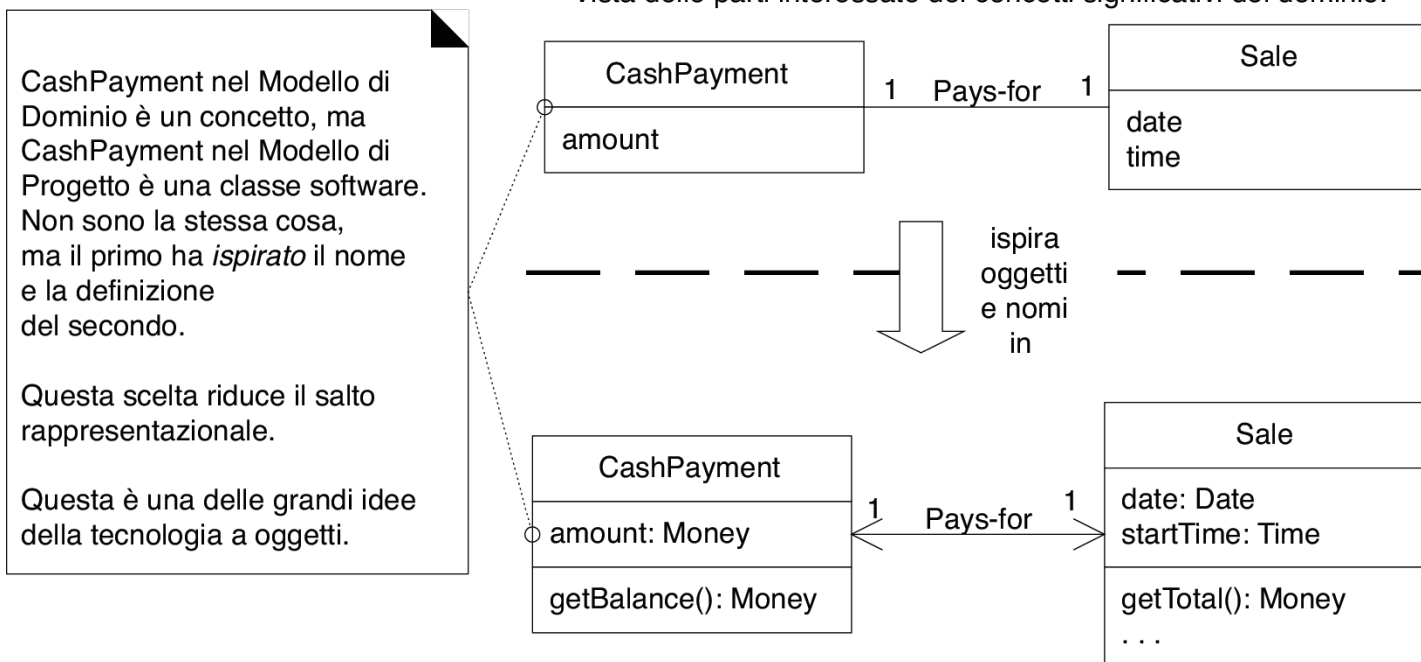
L'organizzazione in strati e package logici guida l'organizzazione del codice

```
/** Strato UI */
com.mycompany.nextgen.ui.swing
com.mycompany.nextgen.ui.web
/** Strato DOMAIN */
// package specifici del progetto NextGen
com.mycompany.nextgen.domain.sales
com.mycompany.nextgen.domain.payments
/** Strato TECHNICAL SERVICES */
// il nostro strato di persistenza (accesso alla base di dati)
com.mycompany.service.persistence
// terze parti
org.apache.log4j
org.apache.soap.rpc
/** Strato FOUNDATION */
// package foundation creati dal nostro team
com.mycompany.util
```

Come va progettata la logica applicativa con gli oggetti?

- Creare oggetti software con nomi e informazioni simili al dominio del mondo reale e assegnare a essi responsabilità della logica applicativa
- Un oggetto software di questo tipo è chiamato un **oggetto di dominio**
 - *Rappresenta una cosa nello spazio del dominio del problema e ha una logica applicativa o di business correlata (un oggetto Sale e in grado di calcolare il suo totale)*
- Progettando gli oggetti in questo modo si arriva a uno strato della logica applicativa che può essere chiamato, in modo più preciso, **strato del dominio** dell'architettura
 - *Contiene oggetti di dominio per gestire la logica applicativa*

Qual è la relazione tra lo strato del dominio e il modello di dominio?



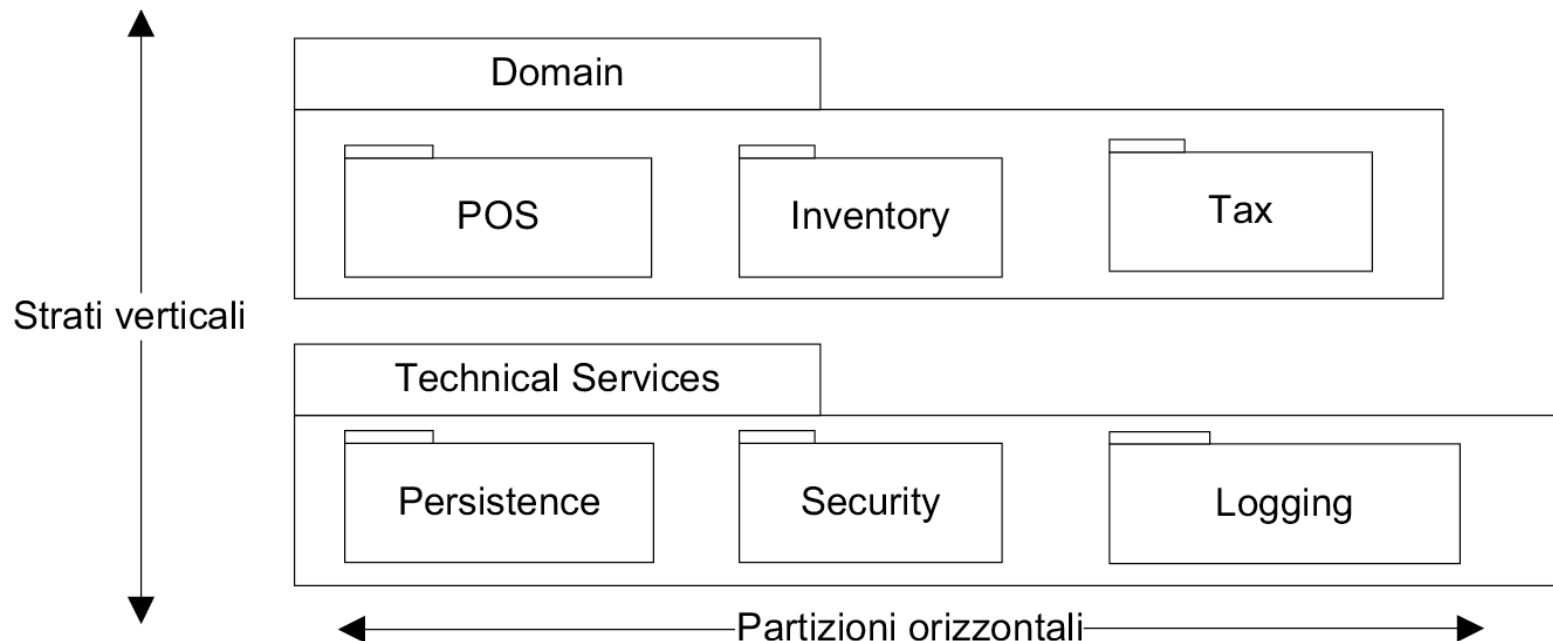
Modello di Progetto di UP

Lo sviluppatore orientato agli oggetti ha tratto ispirazione dal dominio del mondo reale per la creazione di classi software.

Pertanto, il salto rappresentazionale tra il modo in cui le parti interessate concepiscono il dominio e la sua rappresentazione nel software è stato tenuto basso.

Definizione: livelli, strati e partizioni

- **Livello (tier):** solitamente indica un nodo fisico di elaborazione
- **Strato (layer):** una sezione verticale dell'architettura
- **Partizione (partition):** una divisione orizzontale di sottosistema di uno strato



Principio de separazione Modello-Vista

- Gli oggetti di dominio (**modello**) non devono essere connessi o accoppiati direttamente agli oggetti UI (**vista**)
 - *Non permettere a un oggetto di dominio Sale di avere un riferimento a un oggetto finestra JFrame di Java Swing*
- Non mettere la logica applicativa nei metodi di un oggetto è l'interfaccia utente
 - Gli oggetti UI dovrebbero solo inizializzare gli elementi dell'interfaccia utente, ricevere eventi UI, e delegare le richieste di logica applicativa agli oggetti non UI

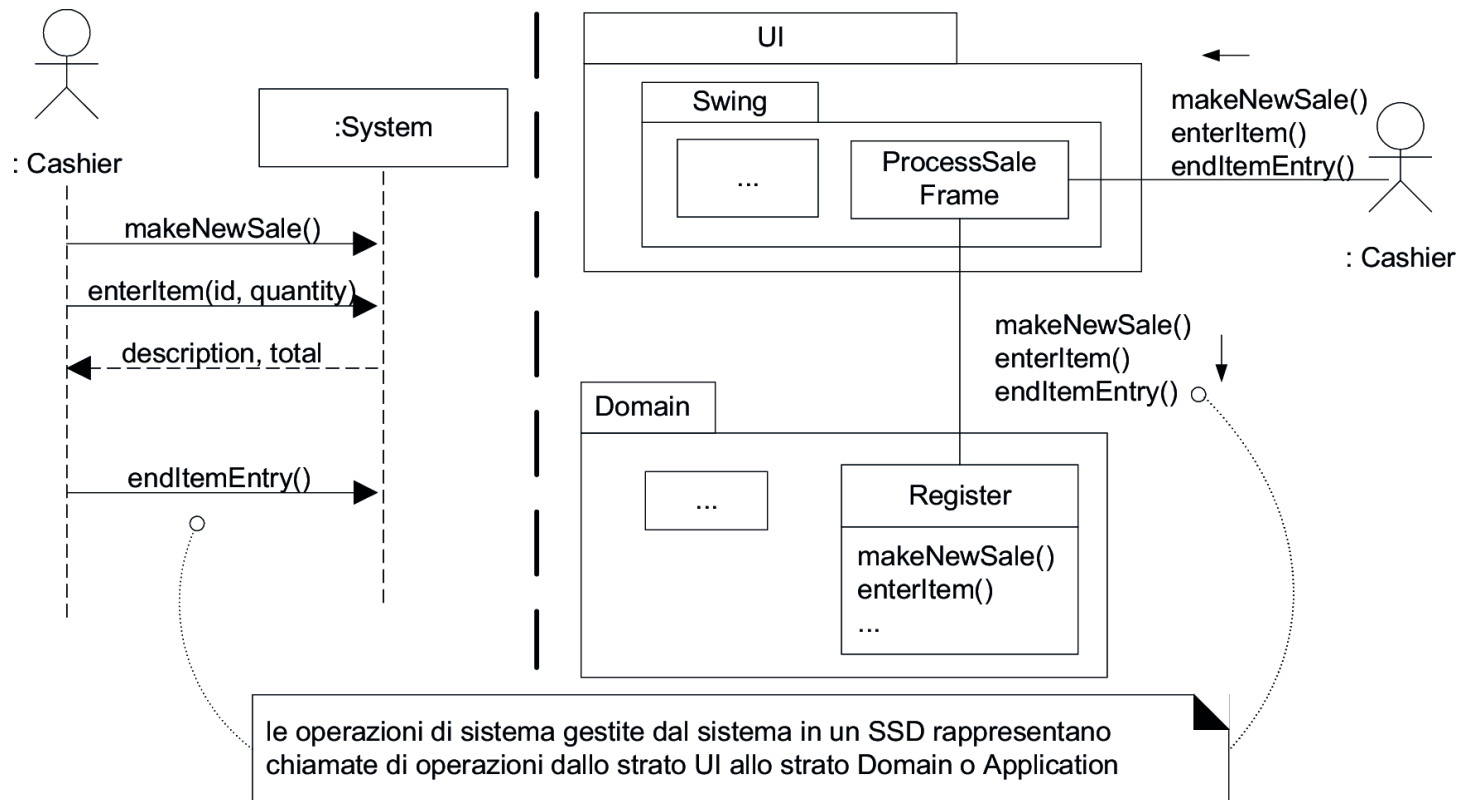
Pattern Observer

Un rilassamento legittimo di questo principio è il pattern Observer:

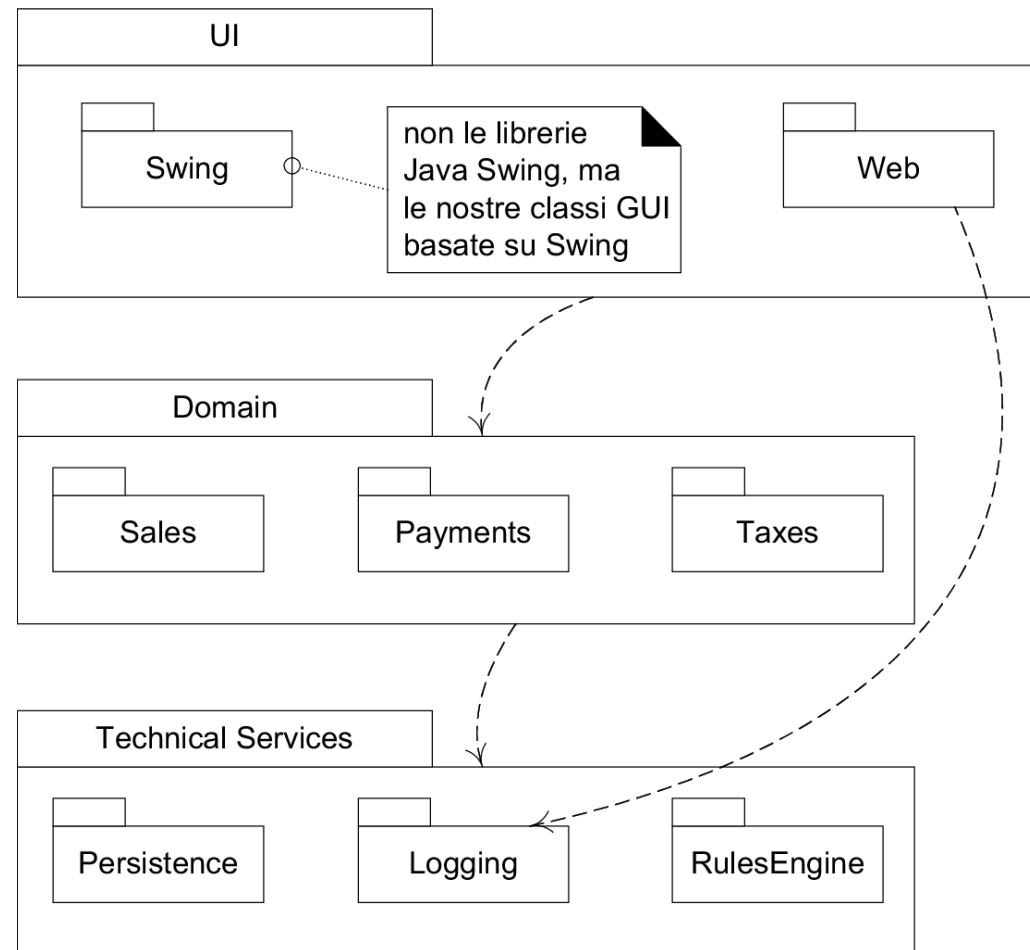
- Gli oggetti del dominio inviano messaggi a oggetti della UI, visti però solo indirettamente, in termini di un'interfaccia come *PropertyListener*
- L'oggetto di dominio non sa che quell'oggetto della UI è un oggetto UI
 - *Non conosce la sua classe UI concreta*
 - *Sa solo che l'oggetto implementa l'interfaccia (non UI) PropertyListener*

Legame tra SSD, operazioni di sistema e strati

I messaggi inviati dallo strato UI allo strato del dominio sono i messaggi mostrati negli SSD, come *enterItem*



Architettura logica e diagramma dei package per NextGen



Architettura logica per Monopoly

- semplice architettura a strati
 - *UI*
 - *dominio*
 - *servizi tecnici*

Analisi e Progettazione del Software

Verso la progettazione a oggetti

Oliviero Riganelli

Come fanno gli sviluppatori a progettare gli oggetti?

1. **Codifica.** Progettare mentre si codifica (in Java, C#, ..), possibilmente insieme a TDD e refactoring
2. **Disegno, poi codifica.** Disegnare alcuni diagrammi UML su una lavagna o con uno strumento CASE per UML per poi passare al punto 1 usando un IDE “testuale” (es. intellij)
3. **Solo disegno.** In qualche modo, lo strumento genera ogni cosa dai diagrammi.

Questa lezione introduce la progettazione oggetti e il *disegno leggero* prima della codifica

Agile Modeling e il disegno leggero di UML

- Ridurre il costo aggiuntivo del disegno e modellare per comprendere e comunicare, anziché documentare
- La modellazione agile comprende anche le seguenti pratiche
 - *Modellare insieme agli altri*
 - *Creare diversi modelli in parallelo*
 - Per esempio, 5 min per i diagrammi di interazione su una lavagna, poi 5 min per i diagrammi delle classi corrispondenti su un'altra lavagna

Strumenti CASE per UML

- La modellazione agile alla parete e l'utilizzo di uno strumento CASE per UML possono essere pratiche complementari
- Si scelga uno strumento CASE per UML che si integra con un IDE diffuso come Eclipse.
- Si scelga uno strumento UML in grado di eseguire il reverse-engineering non solo per i diagrammi delle classi, ma anche i diagrammi di interazione

Quanto tempo dedicare al disegno UML prima della codifica

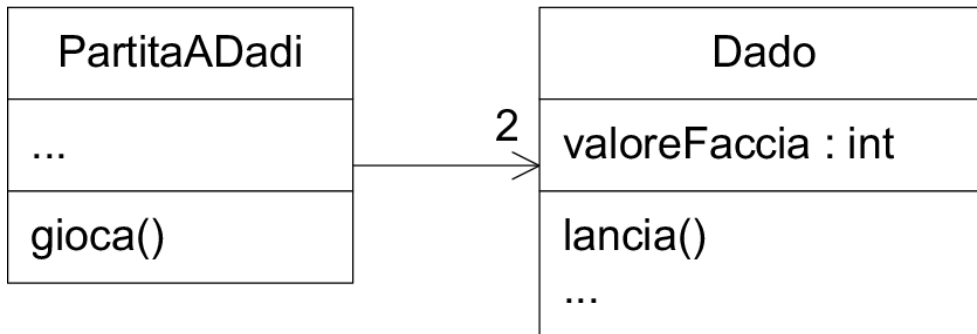
Per un'iterazione di 3 settimane

- Si dedichino alcune ore o al massimo un giorno al disegno di UML
- Si passi alla codifica per il resto dell'iterazione
- Nel corso dell'iterazione possono esserci altre sessioni di disegno più brevi
- Prima di ogni sessione di modellazione successiva, si esegua il reverse-engineering e si faccia riferimento a questi diagrammi nel corso della sessione successiva

Modellazione statica e dinamica

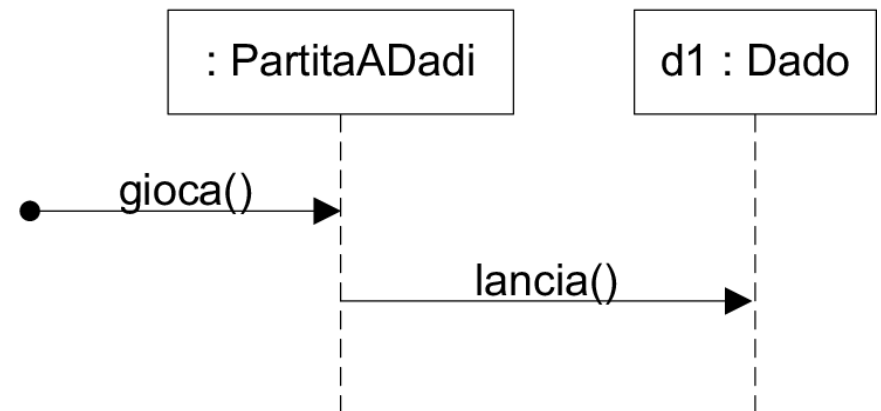
- Ci sono 2 tipi di modelli per gli oggetti: statici e dinamici
- Creare questi modelli in parallelo

Modello statico



Un diagramma delle classi di UML.

Modello dinamico



Un diagramma di sequenza di UML.

Modellazione statica e dinamica

Modelli dinamici

- diagrammi di sequenza e comunicazione
- i più importanti e difficili da creare
- Si applicano
 - La *progettazione guidata dalle responsabilità*
 - I principi **GRASP**

Modelli statici

- diagrammi delle classi
- utili come sintesi e come base per la struttura del codice

Si dedichi abbastanza tempo alla creazione dei diagrammi di iterazione

L'importanza della capacità di progettazione a oggetti rispetto alla conoscenza della notazione UML

- Una notazione è utile, ma ciò che è importante è sapere
 - *come pensare e progettare a oggetti*
 - *applicare le best practice della progettazione a oggetti dei pattern*
- Il disegno con UML è semplicemente una conseguenza delle decisioni di progetto
- la progettazione a oggetti richiede la conoscenza di
 - *principi di assegnazione di responsabilità*
 - *design pattern*

Altre tecniche di progettazione oggetti: Schede CRC

Class, Responsibility, Collaboration

<u>Nome classe</u> - Responsabilità 1 - Responsabilità 2 - Responsabilità 3	Collaboratore 1 Collaboratore 2
---	------------------------------------

Quattro esempi di schede CRC

